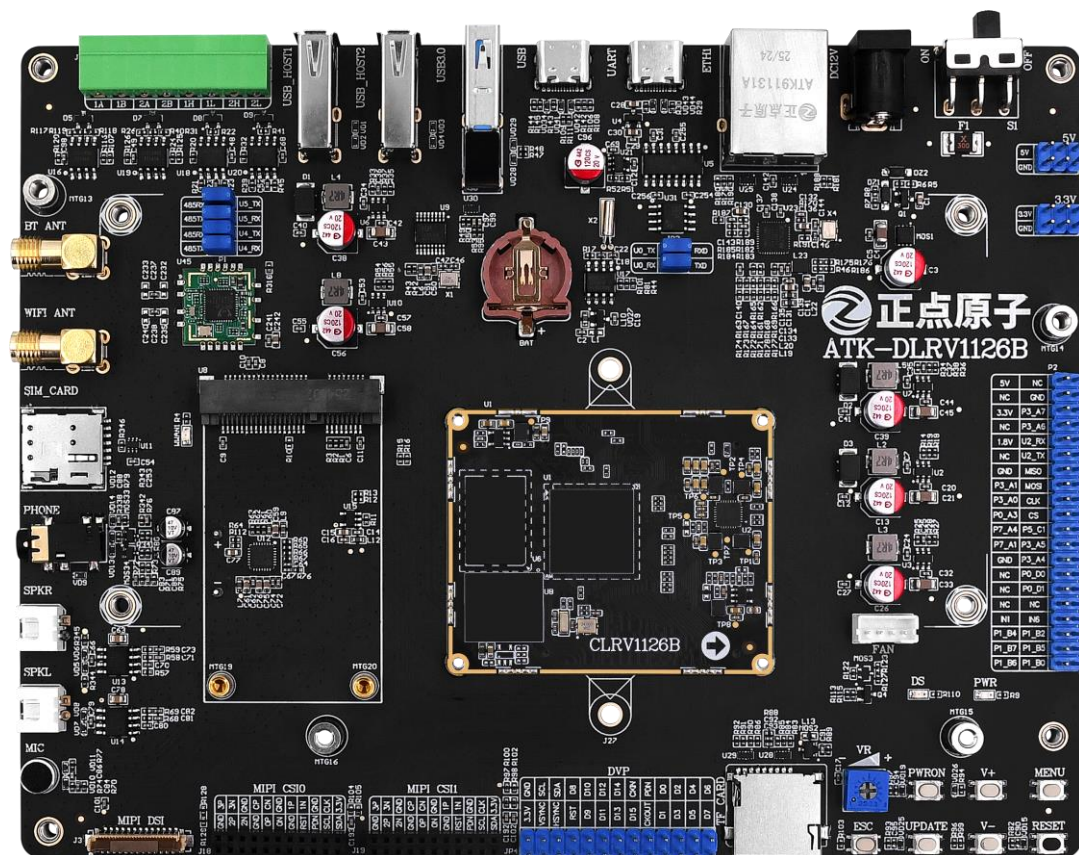


ATK-DLRV1126B

SDK Development Manual

V1.0



1. Shopping:TMALL: <https://zhengdianyuanzi.tmall.com>TAOBAO: <https://openedv.taobao.com>**2. Download**Address: <http://www.openedv.com/docs/index.html>**3. FAE**Website : www.alientek.comForum : <http://www.openedv.com/forum.php>Videos : www.yuanzige.com

Fax : +86 - 20 - 36773971

Phone : +86 - 20 - 38271790



Disclaimer

The product specifications and instructions mentioned in this document are for reference only and subject to update without prior notice; Unless otherwise agreed, this document is intended as a product guide only, and none of the representations made herein constitutes a warranty of any kind. The copyright of this document belongs to Guangzhou Xingyi Electronic Technology Co., LTD. Without the written permission of the company, any unit or individual shall not be used for profit-making purposes in any way of dissemination.

In order to get the latest version of product information, please regularly visit the download center or contact the customer service of Taobao ALIENTEK flagship store. Thank you for your tolerance and support.

Revision History:

Version	Version Update Notes	Responsible person	Proofreading	Date
V1.0	release officially	ALIENTEK	ALIENTEK	2025.12.04

Catalogue

Brief	1
Chapter 1.Install the Ubuntu system	2
1.1 SDK Source Code Installation	2
1.2 Host Environment Configuration.....	3
1.3 Special Precautions	4
1.4 Installing Dependent Libraries and Tools on the Host.....	4
Chapter 2.SDK Compilation	5
2.1 Introduction to the SDK Directory	5
2.2 A First Look at build.sh	5
2.3 Selecting the SDK Board-level Configuration File.....	8
2.4 Fully Automatic SDK Compilation	9
2.5 Generated Image Files	10
Chapter 3.SDK Development	12
3.1 SDK Board-level Configuration File Development.....	12
3.1.1 Location of Board-level Configuration Files	12
3.1.2 Detailed Explanation of Board-level Configuration Files.....	12
3.1.3 How to Modify the Board-level Configuration File.....	13
3.2 U-Boot Development	13
3.2.1 How to Modify the Defconfig.....	13
3.3 Kernel Development	14
3.3.1 How to Modify the Defconfig.....	14
3.3.2 Modifying the DTS	14
3.4 Buildroot Development.....	15
3.4.1 How to Modify the Defconfig.....	15
3.4.2 How to Add Custom Firmware	15
3.5 How to Flash the Firmware.....	16

Brief

This document describes the SDK compilation and usage of the ATK-DLRV1126B. If you are not familiar with concepts such as the kernel, U-boot, and Buildroot, please learn about them first. This document will not cover too many conceptual explanations. This means users are required to have a certain foundation, which is mandatory.

Because Rockchip's SDK involves a great many components, for beginners, understanding just the Buildroot system, U-boot, or the kernel can take a long time. You can refer to the tutorials for the ALIENTEK Alpha Development Board, which are very detailed.

If you really have little foundational knowledge, the author recommends that you at least master basic Ubuntu commands before developing with this SDK, such as file decompression, file creation, file modification, and so on.

For users who do not need additional functional customization, there is no need to develop the kernel, U-boot, or Buildroot. It is more convenient to directly conduct application development based on our pre-built SDK.

If you need to add extra functions, you will need to learn how to modify our SDK. Now, let's start developing the SDK!

Chapter 1. Install the Ubuntu system

This chapter describes the SDK source code installation and relevant precautions.

1.1 SDK Source Code Installation

Please copy the Linux SDK compressed package file from the cloud disk to the home directory of your Ubuntu virtual machine.

The specific steps are as follows:

Open the development board resource package and navigate to the following path:

Development Board Cloud Disk Drive B → Development Environment & SDK → 02. ATK-DLRV1126B Development Board SDK → 01. linux6.1_sdk

Copy the file `atk_dlr1126b_linux6.1_sdk_release_v1.0.0_20251202.tar.gz` to the home directory of your Ubuntu virtual machine.

File Name Explanation

- v1.0.0 indicates version 1.0.0 released by ALIENTEK
- 20251202 indicates the release date (December 2, 2025)

If subsequent versions are updated, please copy the corresponding latest SDK source code file from the cloud disk according to the actual situation.

After copying is completed, you can perform SDK decompression and compilation in the home directory.

```
alientek@alientek:~$ ls atk_dlr1126b_linux6.1_sdk_release_v1.0.0_20251202.tar.gz
atk_dlr1126b_linux6.1_sdk_release_v1.0.0_20251202.tar.gz
alientek@alientek:~$
```

If you are not familiar with file copy operations under the Ubuntu system, please refer to relevant documents to learn the operation methods first.

For details, please refer to:

Development Board Cloud Disk Drive A → Basic Materials → 10. User Manuals → 02. Auxiliary Documents → 【ALIENTEK】Ubuntu&Windows&Linux Development Board File Transfer Reference Manual

This manual details various file transfer methods between Windows, Ubuntu and the development board, including GUI and command-line operations, suitable for users at all levels.

```
tar xf atk_dlr1126b_linux6.1_sdk_release_v1.0.0_20251202.tar.gz
```

Decompression takes several tens of seconds.

```
ls -l
```

```
alientek@alientek:~$ tar xf atk_dlr1126b_linux6.1_sdk_release_v1.0.0_20251202.tar.gz
alientek@alientek:~$ ls atk_dlr1126b_linux6.1_sdk_release_v1.0.0_20251202
使用说明.md  buildroot  README.md  repo.sh
alientek@alientek:~$
```

```
cd atk_dlr1126b_linux6.1_sdk_release_v1.0.0_20251202
```

```
ls -a
```

```
alientek@alientek:~$ cd atk_dlr1126b_linux6.1_sdk_release_v1.0.0_20251202
alientek@alientek:~/atk_dlr1126b_linux6.1_sdk_release_v1.0.0_20251202$ ls -a
.  ..  使用说明.md  buildroot  README.md  .repo  repo.sh
alientek@alientek:~/atk_dlr1126b_linux6.1_sdk_release_v1.0.0_20251202$
```

Source Code Checkout Instructions

Now we will start checking out the SDK source code.

As you can see from the image above, you will notice that the source code directory contains a hidden folder named `.repo`. This is because Rockchip officially uses the **repo** tool to manage its multi-repository Git project structure.

Repo is a tool developed by Google for managing multiple Git repositories. It is especially suitable for large projects with numerous submodules, such as the Linux kernel and Android system. Through the `.repo` directory, the repo tool can uniformly manage the versions, dependencies, and synchronization status of all sub-repositories, greatly simplifying the maintenance process of multi-repository projects.

Execute the `./repo.sh` script to check out the source code.

```
./repo.sh # Checkout Script
```

The content of the **./repo.sh checkout script** is as follows. You can also directly copy the command below to check out the SDK source code.

```
.repo/repo/repo sync -l -j10
```

Note: The "-l" here is the lowercase letter "L" (not the number 1).

```
alientek@alientek:~/atk_dlr1126b_linux6.1_sdk_release_v1.0.0_20251202$ ./repo.sh
正在更新文件: 100% (20312/20312), 完成.
正在更新文件: 100% (84562/84562), 完成.
正在更新文件: 100% (858/858), 完成.
正在更新文件: 100% (5939/5939), 完成.
正在更新文件: 100% (5722/5722), 完成.
正在更新文件: 100% (8550/8550), 完成.
正在更新文件: 100% (633/633), 完成.
正在更新文件: 100% (429/429), 完成.
正在更新文件: 100% (324/324), 完成.
正在更新文件: 100% (41800/41800), 完成.
正在更新文件: 100% (3612/3612), 完成.
正在更新文件: 100% (210/210), 完成.
正在更新文件: 100% (48/48), 完成.
正在更新文件: 100% (63/63), 完成.
正在更新文件: 100% (258/258), 完成.
正在更新文件: 100% (13994/13994), 完成.
正在更新文件: 100% (581/581), 完成. k/u-boot正在更新文件: 97% (566/581)
正在更新文件: 100% (289/289), 完成.
正在更新文件: 100% (1195/1195), 完成.
正在更新文件: 100% (6395/6395), 完成.
Checking out: 100% (45/45), done in 2m48.595s
repo sync has finished successfully.
alientek@alientek:~/atk_dlr1126b_linux6.1_sdk_release_v1.0.0_20251202$
```

After checking out the source code, you can delete

```
.atk_dlr1126b_linux6.1_sdk_release_v1.0.0_20251202.tar.gz
```

1.2 Host Environment Configuration

System Environment Requirements

Ubuntu Version Requirements:

This SDK development environment is developed and tested based on **Ubuntu 20.04 LTS**. To ensure successful compilation, you are recommended to use the same version.

If you are familiar with the Ubuntu system, you may try other newer Ubuntu versions, but you will need to resolve any compatibility issues that may arise on your own (generally minor issues). Beginners should strictly configure the environment according to the specified version.

Disk Space Requirements:

Disk Space Requirements:

- Minimum requirement: available disk capacity ≥ 100 GB
- Recommended configuration: 150 GB or above
- More available space ensures a smooth compilation process and avoids compilation failure due to insufficient space.

Virtual Machine Resource Configuration:

- Memory: recommended allocation ≥ 8 GB (the more, the better)
- Minimum attempt: 4 GB (swap space required)
- Processor cores: allocate as many cores as possible

Performance Optimization Tips:

If virtual machine memory is limited, you can refer to the following manual to configure swap space for a better compilation experience:

Development Board Cloud Disk A → Basic Documents → 10. User Manuals → 02. Auxiliary Documents → **【ALIENTEK】Ubuntu Swap Space Expansion Reference Manual**

Compilation Time Estimation

Under the condition that no errors occur during compilation, a full compilation usually takes **1.5 to 2 hours**.

The actual compilation time is affected by the following factors:

- Disk read/write speed (SSD is significantly better than HDD)
- Number of CPU cores and main frequency
- Available system memory

As for compilation time, it normally takes **2 to 3 hours** without errors. It is related to disk access speed and CPU performance.

1.3 Special Precautions

- Do not compile or decompress the SDK source code using the root user!
- Do not decompress the SDK source code with sudo privileges!
- The source code must not be decompressed to a path containing Chinese characters!

1.4 Installing Dependent Libraries and Tools on the Host

```
sudo apt-get update                                # First, update the software sources.
sudo apt-get install git ssh make gcc libssl-dev \
liblz4-tool expect expect-dev g++ patchelf chrpath gawk texinfo chrpath \
diffstat binfmt-support qemu-user-static live-build bison flex fakeroot \
cmake gcc-multilib g++-multilib unzip device-tree-compiler ncurses-dev \
bzip2 expat gpgv2 cpp-aarch64-linux-gnu libgmp-dev \
libmpc-dev bc python-is-python3 python2 gettext libc6-dev libncurses-dev rsync
sudo apt-get install gettext                    # The Ubuntu 20 virtual machine we provided also n
eeds to have these installed.
sudo apt-get install clang-format                  # For Ubuntu 24, these additional packages need to b
e installed as well.
```

Note: If you use other versions of Ubuntu, please use the corresponding installation commands according to the package names. If you encounter errors during compilation, you may install the corresponding software packages based on the error messages.

Chapter 2. SDK Compilation

This chapter covers the compilation of the SDK.

2.1 Introduction to the SDK Directory

The root directory of the SDK source code contains multiple subdirectories such as **app**, **buildroot**, **device**, **external**, **tools**, **u-boot**, etc. Each directory or its subdirectories corresponds to a Git project. Since the SDK code and related documents are divided into several Git repositories for separate version management (the SDK actually contains multiple Git repositories), they are organized into different directories according to their functions and corresponding modules.

- **app**: Stores upper-layer applications and other C/C++ programs.
- **buildroot**: Root file system developed based on Buildroot.
- **device/rockchip**: Stores board-level configuration files, partition table files (Parameter), as well as scripts and preparation files for compiling and packaging firmware for various chips.
- **docs**: Stores chip module development guides, platform support lists, chip platform-related documents, Linux development guides, etc.
- **external**: Stores required third-party libraries, including audio, video, network, recovery, etc.
- **kernel**: Linux kernel source code (version 6.1).
- **prebuilts**: Stores cross-compilation toolchains.
- **rkbin**: Stores Rockchip-related binary files and tools.
- **rockdev**: Stores compiled output firmware. This folder is generated only after the SDK is compiled.
- **tools**: Stores common tools for Linux and Windows operating systems, including image flashing tools, SD card upgrade and boot making tools, batch flashing tools, etc. For example, the RKDevTool and Linux_Upgrade_Tool mentioned earlier are stored in this directory.
- **u-boot**: U-Boot source code.

Note: There are also some symbolic link files, which are not listed here.

2.2 A First Look at build.sh

build.sh is a compilation script provided by Rockchip. Using this script, users can conveniently and quickly build various image files and package images. It supports one-click fully automatic compilation of the entire SDK, as well as separate compilation of U-Boot, Linux Kernel, buildroot, etc., which is very convenient!

The build.sh script is actually a symbolic link that points to the actual file:

device/rockchip/common/build.sh.

You can view the usage of the build.sh script by running the following command:

```
./build.sh -h
```

Usage examples:

For example, if you need to clean the kernel, run:

```
./build.sh clean-kernel
```

You can learn more usage methods as needed.

Usage: build.sh [OPTIONS]

Available options:

chip[:<chip>[:<config>]]	choose chip
defconfig[:<config>]	choose defconfig
*_defconfig	switch to specified defconfig
available defconfigs:	
01_atk_dlr1126b_automipi_defconfig	
02_atk_dlr1126b_mipi720x1280_defconfig	
03_atk_dlr1126b_mipi800x1280_defconfig	
04_atk_dlr1126b_mipi1080x1920_defconfig	
05_atk_dlr1126b_automipi_amp_mcu_defconfig	
olddefconfig	resolve any unresolved symbols in .config
savedefconfig	save current config to defconfig
menuconfig	interactive curses-based configurator
config	modify SDK defconfig
config-rootfs-type	modify SDK rootfs type config
config-usb-gadget	modify SDK USB gadget config
config-wifibt	modify SDK Wi-Fi/BT config
config-rootfs-overlay	modify SDK rootfs overlay config
config-overlay	alias of config-rootfs-overlay
config-post-rootfs	modify SDK post rootfs config
config-extra-parts	modify SDK extra partitions config
print-parts	print partitions
list-parts	alias of print-parts
edit-parts	edit raw partitions
misc	pack misc image
kernel-6.1[:dry-run]	build kernel 6.1
kernel[:dry-run]	build kernel
recovery-kernel[:dry-run]	build kernel for recovery
kernel-modules[:<dst dir>:dry-run]	build kernel modules
modules[:<dst dir>:dry-run]	alias of kernel-modules
linux-headers[:<arch>:dry-run]	build linux-headers
kernel-config[:dry-run]	modify kernel defconfig
kconfig[:dry-run]	alias of kernel-config
kernel-make[:<arg1>:<arg2>]	run kernel make
kmake[:<arg1>:<arg2>]	alias of kernel-make
wifibt[:<dst dir>[:<chip>]]	build Wifi/BT
amp	build and pack amp system
buildroot-config[:<config>]	modify buildroot defconfig
bconfig[:<config>]	alias of buildroot-config
buildroot-make[:<arg1>:<arg2>]	run buildroot make
bmake[:<arg1>:<arg2>]	alias of buildroot-make
buildroot-sdk	build the buildroot SDK tarball

bsdk	alias of buildroot-sdk
edit-debian-packages	edit debian package list
rootfs[:<rootfs type>]	build default rootfs
buildroot	build buildroot rootfs
yocto	build yocto rootfs
debian	build debian rootfs
recovery	build recovery
security-createkeys	create keys for security
security-misc	build misc with system encryption key
security-ramboot[:system_image]	build security ramboot
security-system[:system_image]	build security system
security-remote-sign	build remote signed image
loader[:dry-run]	build loader (u-boot)
uboot[:dry-run]	build u-boot
u-boot[:dry-run]	alias of uboot
extra-parts	pack extra partition images
firmware	pack and check firmwares
edit-package-file	edit package-file
edit-ota-package-file	edit package-file for OTA
updateimg	build update image
ota-updateimg	build update image for OTA
all	build images
release[:<subdir>[:<name>]]	release images and build info
all-release[:<subdir>[:<name>]]	build and release images
shell	setup a shell for developing
buildroot-shell	setup a shell for buildroot developing
bshell	alias of buildroot-shell
yocto-shell	setup a shell for yocto developing
yshell	alias of yocto-shell
cleanall	cleanup all
clean-misc	cleanup misc
clean-firmware	cleanup firmware
clean-recovery	cleanup recovery
clean-rootfs	cleanup rootfs
clean-kernel	cleanup kernel
clean-config	cleanup config
clean-security	cleanup security
clean-extra-parts	cleanup extra-parts
clean-updateimg	cleanup updateimg
clean-loader	cleanup loader
clean-amp	cleanup amp
post-rootfs <rootfs dir>	trigger post-rootfs hook scripts
help	display this information

Default option is 'all'.

2.3 Selecting the SDK Board-level Configuration File

This section describes how to select the configuration for the ATK-DLRV1126B Linux SDK.

First, go to the top-level directory of the SDK source code, and run the following command before compilation to specify the board-level configuration file for the SDK.

Enter the top-level directory of the SDK source code and execute the command below to select the board-level configuration file.

(Beginners may ask: Why do this step?

Answer: Because we need to tell the SDK what kind of image to build. Different configurations correspond to different functions. For example, if you need to verify AMP, select the configuration with AMP.)

```
./build.sh lunch
```

After executing the above command, you will be asked to enter a serial number to select the board-level configuration file. The function of this board-level file is to configure different device trees, different kernel configurations, and different file system types to adapt to various types of development boards.

```

alientek@alientek:~/atk_dlr1126b_linux6.1_sdk_release_v1.0.0_20251202$ ./build.sh lunch

##### Rockchip Linux SDK #####
Manifest: atk-dlr1126b_linux6.1_release_v1.0.0_20251202.xml
Log colors: message notice warning error fatal

    板级配置选择指南
    +-----+
    | atk_dlr1126b | 正点原子RV1126B |
    | mipi720x1280 | 720x1280 分辨率MIPI屏 |
    | mipi800x1280 | 800x1280 分辨率MIPI屏 |
    | mipi1080x1920 | 1080x1920 分辨率MIPI屏 |
    | automipi | 自动识别正点原子MIPI屏 |
    | amp_mcu | AMP MCU镜像 |
    | defconfig | 配置文件 |
    +-----+

根据配置解释, 输入对应的序号, 如输入数字1
若不清楚配置, 请随意选一个对应项的数字, 编译成功后可再重新切换配置再编译
1. 无屏幕用户: 选择任意一个分辨率配置或automipi配置
2. 有正点原子MIPI屏: 选择对应分辨率配置
3. automipi: 编译正点原子所有MIPI屏设备树, 自动识别屏幕
4. SDK更多开发资料参考/10、用户手册、开发文档/xxx_SDK开发指南
5. amp_mcu: mcu加载amp.img镜像, 用于验证mcu (多核异构)

Log saved at /home/alientek/atk_dlr1126b_linux6.1_sdk_release_v1.0.0_20251202/output/sessions/2025-12-03_15-37-41
Pick a defconfig:
1. 01_atk_dlr1126b_automipi_defconfig
2. 02_atk_dlr1126b_mipi720x1280_defconfig
3. 03_atk_dlr1126b_mipi800x1280_defconfig
4. 04_atk_dlr1126b_mipi1080x1920_defconfig
5. 05_atk_dlr1126b_automipi_amp_mcu_defconfig
Which would you like? [1]: 1
  
```

Enter the serial number to view the printed board-level configuration selection guide.

The configuration for the industrial-grade RV1126BJ is identical to that of the commercial-grade RV1126BJ; no distinction is required. Users can refer to the printed board-level configuration instructions. If ALIENTEK users are unsure which resolution MIPI screen they are using, simply compile with **01_atk_dlr1126b_automipi_defconfig**. This configuration automatically detects the MIPI screens sold in the ALIENTEK store.

Other configurations, such as **02_atk_dlr1126b_mipi720x1280_defconfig**, are only for MIPI screens with a resolution of 720x1280 and do not support automatic MIPI screen detection. Users can

directly modify the MIPI screen parameters in this configuration to adapt to their own MIPI screen or other peripheral configurations, which is very convenient.

Users can freely choose any configuration from 1 to 5 for compilation. During testing, you can run `./build.sh` again to select and switch to other configurations **without needing to clean and recompile**, which is very convenient!

2.4 Fully Automatic SDK Compilation

After selecting the board-level configuration in the previous step (this is required and only needs to be done once; you can directly run `./build.sh` next time), execute the following command to start compiling the SDK. Run the command below to compile everything.

`./build.sh`

```
allientek@allientek:~/atk_dlr1126b_linux6.1_sdk_release_v1.0.0_20251202$ ./build.sh
##### Rockchip Linux SDK #####
Manifest: atk-dlr1126b_linux6.1_release_v1.0.0_20251202.xml
Log colors: message notice warning error fatal
Log saved at /home/allientek/atk_dlr1126b_linux6.1_sdk_release_v1.0.0_20251202/output/sessions/2025-12-03_15-52-30
Using last kernel version(6.1)
=====
Final configs
=====
RK_BOOT_FIT_ITS=/home/allientek/atk_dlr1126b_linux6.1_sdk_release_v1.0.0_20251202/device/rockchip/.chip/boot.its
RK_BOOT_IMG=boot.img
RK_BUILDROOT=y
RK_BUILDROOT_CFG=allientek_rv1126b
RK_CHIP=rv1126b
RK_CHIP_FAMILY=rv1126b
RK_DEBIAN=y
RK_DEBIAN_ARCH=arm64
RK_DEBIAN_BOOKWORM=y
RK_DEBIAN_LINUX_HEADERS=y
RK_DEBIAN_MIRROR=mirrors.ustc.edu.cn
RK_DEBIAN_PASSWORD=*****
RK_DEBIAN_USER=linaro
RK_DEBIAN_VERSION=bookworm
RK_DEFCONFIG=01_atk_dlr1126b_autompi_defconfig
RK_EXTRA_PARTITION_NUM=2
RK_EXTRA_PARTITION_STR=auto:oem:auto:ext4:defaults:normal:auto:@auto:userdata:auto:ext4:defaults:normal:auto:@@
RK_KERNEL=y
RK_KERNEL_ARCH=arm64
```

After about 3 hours, the compilation is complete, as shown in the figure below.

```

backup    RESERVED
rootfs    rootfs.img
oem       oem.img
userdata  userdata.img
Packing /home/alientek/atk_dlr1126b_linux6.1_sdk_release_v1.0.0_20251202/output/firmware/update.img for update...
Android Firmware Package Tool v2.29
----- PACKAGE -----
Add file: ./package-file
package-file,Add file: ./package-file done,offset=0x800,size=0xe1,userspace=0x1
Add file: ./parameter.txt
parameter,Add file: ./parameter.txt done,offset=0x1000,size=0x229,userspace=0x1,flash_address=0x00000000
Add file: ./MiniLoaderAll.bin
bootloader,Add file: ./MiniLoaderAll.bin done,offset=0x1800,size=0x6e1c0,userspace=0xdd
Add file: ./uboot.img
uboot,Add file: ./uboot.img done,offset=0x70000,size=0x400000,userspace=0x800,flash_address=0x00004000
Add file: ./misc.img
misc,Add file: ./misc.img done,offset=0x470000,size=0xc000,userspace=0x18,flash_address=0x00006000
Add file: ./boot.img
boot,Add file: ./boot.img done,offset=0x47c000,size=0x27d3800,userspace=0x4fa7,flash_address=0x00008000
Add file: ./recovery.img
recovery,Add file: ./recovery.img done,offset=0x2c4f800,size=0x2f7a400,userspace=0x5ef5,flash_address=0x00028000
Add file: ./rootfs.img
rootfs,Add file: ./rootfs.img done,offset=0x5bca000,size=0x4ed00000,userspace=0x9da00,flash_address=0x00078000
Add file: ./oem.img
oem,Add file: ./oem.img done,offset=0x548ca000,size=0x1157000,userspace=0x22ae,flash_address=0x00c78000
Add file: ./userdata.img
userdata,Add file: ./userdata.img done,offset=0x55a21000,size=0x800000,userspace=0x1000,flash_address=0x00cb8000
Add CRC...
Make firmware OK!
----- OK -----
*****rkImageMaker ver 2.29*****
Generating new image, please wait...
Writing head info...
Writing boot file...
Writing firmware...
Generating MD5 data...
MD5 data generated successfully!
New image generated successfully!

Run 'make edit-package-file' if you want to change the package-file.

Running 90-updateimg.sh - build_updateimg succeeded.
alientek@alientek:~/atk_dlr1126b_linux6.1_sdk_release_v1.0.0_20251202$

```

2.5 Generated Image Files

The generated image files are located in the **SDK/rockdev** folder. The files under rockdev are symbolic links, as shown in the figure below.

```

alientek@alientek:~/atk_dlr1126b_linux6.1_sdk_release_v1.0.0_20251202$ ls rockdev/ -l
总用量 0
lrwxrwxrwx 1 alientek alientek 25 12月 3 16:00 boot.img -> ../../kernel-6.1/boot.img
lrwxrwxrwx 1 alientek alientek 45 12月 3 15:57 MiniLoaderAll.bin -> ../../u-boot/rv1126b_spl_loader_v1.06.105.bin
lrwxrwxrwx 1 alientek alientek 11 12月 3 15:56 misc.img -> ../misc.img
lrwxrwxrwx 1 alientek alientek 22 12月 3 18:35 oem.img -> ../extra-parts/oem.img
lrwxrwxrwx 1 alientek alientek 50 12月 3 18:35 parameter.txt -> ../../device/rockchip/chips/rv1126b/parameter.txt
lrwxrwxrwx 1 alientek alientek 23 12月 3 18:35 recovery.img -> ../recovery/ramboot.img
lrwxrwxrwx 1 alientek alientek 58 12月 3 18:06 rootfs.img -> ../../buildroot/output/alientek_rv1126b/images/rootfs.ext2
lrwxrwxrwx 1 alientek alientek 22 12月 3 15:57 uboot.img -> ../../u-boot/uboot.img
lrwxrwxrwx 1 alientek alientek 26 12月 3 19:55 update.img -> ../update/Image/update.img
lrwxrwxrwx 1 alientek alientek 27 12月 3 18:35 userdata.img -> ../extra-parts/userdata.img
alientek@alientek:~/atk_dlr1126b_linux6.1_sdk_release_v1.0.0_20251202$

```

Introduction to the image files is shown in the table below:

Image Name	Function
uboot.img	uboot.img is an image in FIT format, which is composed of multiple merged images, including the trust image (ARM Trusted Firmware + OP-TEE OS), the u-boot image, and the u-boot dtb. These images are packaged into a single uboot.img when U-Boot is compiled. uboot.img is flashed into the u-boot partition of the development board.
boot.img	boot.img is also an image in FIT format, which is composed of multiple merged images, including the kernel image, kernel DTB, and the resource image resource.img. boot.img is flashed into the boot partition of the development board.

MiniLoaderAll.bin	This image is a section of Loader code that runs before U-Boot on the RK platform (i.e., an earlier-stage Loader than U-Boot). MiniLoaderAll.bin consists of two parts: TPL and SPL. TPL is used to initialize DDR and runs in SRAM, while SPL runs in DDR and is mainly responsible for loading and booting uboot.img.
misc.img	It contains BCB (Bootloader Control Block) information, and this image will be flashed to the misc partition of the development board. The misc partition is a very important partition that stores the BCB data block, which is mainly used for communication between the Android/Linux system, U-Boot, and recovery.
oem.img	For factory use, used to store the manufacturer's APPs or data. This image will be flashed to the oem partition of the development board and mounted to the /oem directory after the system boots.
parameter.txt	A text file that serves as the partition table file for the RK platform (which records the partition names as well as their start and end addresses). When flashing images, the parameter.txt file does not need to be flashed to the Flash; instead, the system reads its information to define the partitions.
recovery.img	Recovery mode image: recovery.img is used to enter recovery mode and is flashed to the recovery partition of the development board. Recovery mode is a mode used for device repair, upgrades, and updates. recovery.img is also a FIT format image, composed of multiple merged images, including ramdisk (the root filesystem mounted when entering recovery mode), kernel image (the kernel image booted when entering recovery mode), kernel DTB, and resource.img.
rootfs.img	The root filesystem image for normal boot mode, which contains a large number of library files, executable files, etc. rootfs.img is flashed to the rootfs partition of the development board.
userdata.img	For user use, it can be used to store users' apps or data. This image will be flashed to the userdata partition of the development board and mounted to the /userdata directory after the system boots.
update.img	Images such as uboot.img, boot.img, MiniLoaderAll.bin, parameter.txt, misc.img, rootfs.img, oem.img, userdata.img, and recovery.img are packaged into a single update.img firmware.

Now that we have finished compiling the SDK, you can use **RKDevTool** to flash the firmware by referring to the factory image flashing document in the user manual/auxiliary documents under the cloud disk resources.

Chapter 3. SDK Development

This chapter will guide you through rapid SDK development.

3.1 SDK Board-level Configuration File Development

3.1.1 Location of Board-level Configuration Files

In Section 2.3, we selected the board-level configuration files. These board-level configuration files are located under <SDK>/device/rockchip/rv1126b/, as shown in the figure below.

```
alientek@alientek:~/atk_dlr1126b_linux6.1_sdk_release_v1.0.0_20251202$ ls device/rockchip/rv1126b
01_atk_dlr1126b_automipi_defconfig      amp_mcu.its      fastboot-boot.its      parameter.txt
02_atk_dlr1126b_mipi720x1280_defconfig  boot-32.its      fastboot-rootfs.its    yocto-rv1126b.conf
03_atk_dlr1126b_mipi800x1280_defconfig  boot4recovery-32.its overlays
04_atk_dlr1126b_mipi1080x1920_defconfig boot4recovery.its parameter-amp.txt
05_atk_dlr1126b_automipi_amp_mcu_defconfig boot.its          parameter-robot.txt
alientek@alientek:~/atk_dlr1126b_linux6.1_sdk_release_v1.0.0_20251202$
```

3.1.2 Detailed Explanation of Board-level Configuration Files

We will take the board-level configuration file device/rockchip/rv1126b/02_atk_dlr1126b_mipi720x1280_defconfig as an example for explanation.

The contents of 02_atk_dlr1126b_mipi720x1280_defconfig are as follows.

```
RK_ROOTFS_PREBUILT_TOOLS=y
RK_UBOOT_SPL=y
RK_UBOOT_CFG="alientek_rv1126b"
RK_KERNEL_CFG="alientek_rv1126b_defconfig"
RK_KERNEL_DTS_NAME="rv1126b-alientek-mipi720x1280"
RK_BUILDROOT_CFG="alientek_rv1126b"
RK_RECOVERY_BASE_CFG="alientek_rv1126b"
RK_ROOTFS_HOSTNAME_CUSTOM=y
RK_ROOTFS_HOSTNAME="ATK-DLRV1126B"
RK_USE_FIT_IMG=y
RK_WIFIBT=n
RK_ROOTFS_INSTALL_MODULES=y
```

- RK_ROOTFS_PREBUILT_TOOLS=y: Enable PREBUILT_TOOLS, i.e., prebuilt tools.
- RK_UBOOT_SPL: Enable SPL, which is generally required.
- RK_UBOOT_CFG="alientek_rv1126b": The defconfig file for U-Boot is alientek_rv1126b_defconfig, located at SDK/u-boot/configs/alientek_rv1126b_defconfig.
- RK_KERNEL_CFG="alientek_rv1126b_defconfig": Similarly, this is the kernel defconfig file, located at SDK/kernel/arch/arm64/configs/alientek_rv1126b_defconfig.
- RK_KERNEL_DTS_NAME="rv1126b-alientek-mipi720x1280": The kernel device tree DTS file is rv1126b-alientek-mipi720x1280.dts, located at SDK/kernel/arch/arm64/boot/dts/rockchip/rv1126b-alientek-mipi720x1280.dts.
- RK_BUILDROOT_CFG="alientek_rv1126b": The defconfig file used by Buildroot, located at SDK/buildroot/configs/alientek_rv1126b_defconfig.

- RK_ROOTFS_HOSTNAME_CUSTOM=y: Enable custom hostname.
- RK_ROOTFS_HOSTNAME="ATK-DLRV1126B": Set the custom hostname to "ATK-DLRV1126B", which modifies /etc/hostname.
- RK_USE_FIT_IMG=y: Use FIT image.
- RK_WIFIBT=n: Disable the custom RK Wi-Fi and Bluetooth driver.
- RK_ROOTFS_INSTALL_MODULES=y: Install kernel modules to /lib/modules in the root filesystem.

3.1.3 How to Modify the Board-level Configuration File

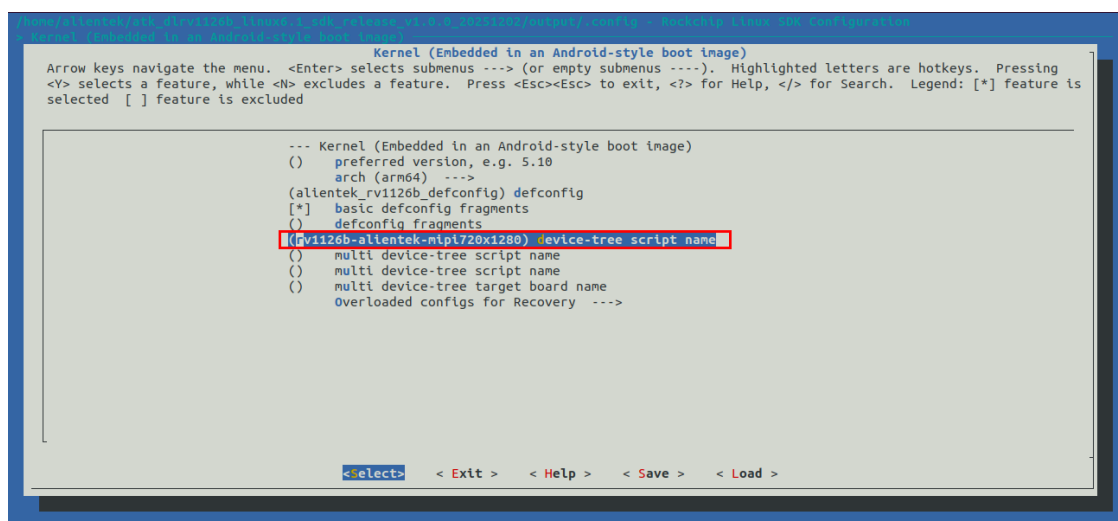
According to the command instructions in Section 2.2, run `./build.sh menuconfig` to modify the board-level configuration file. Modify the corresponding configurations as shown in the figure below.

```
./build.sh 02_atk_dlr1126b_mipi720x1280_defconfig menuconfig
```

Or `./build.sh menuconfig` # You need to run `./build.sh lunch` to select the corresponding board-level configuration file.

For example, you can directly modify the default device tree name if needed. Note that after modifying the device tree name here, the kernel **must** have a DTS device tree with the exact same name!

Note: The board-level configuration file for `auto_mipi` uses a multi-device tree setup. It compiles multiple device trees simultaneously, and automatically identifies the screen via HWID to load the corresponding device tree at boot. Generally, users do not need the `auto_mipi` configuration, as it is exclusive to ALIENTEK for detecting all ALIENTEK MIPI screens. This prevents issues where the same firmware might fail to work with ALIENTEK MIPI screens of different resolutions, which would affect the user experience. Therefore, when creating custom board-level configuration files, users should refer to single-device tree configurations such as `02_atk_dlr1126b_mipi720x1280_defconfig`!



3.2 U-Boot Development

3.2.1 How to Modify the Defconfig

As mentioned in Section 3.1.2, we know the U-Boot configuration file for ALIENTEK RV1126B is `SDK/u-boot/configs/alientek_rv1126b_defconfig`.

We can edit this file SDK/u-boot/configs/alientek_rv1126b_defconfig directly!

Alternatively, some users prefer to configure it using menuconfig. Perform the following steps:

Enter the u-boot directory in the SDK source code.

```
cd SDK/u-boot/
```

Configure menuconfig based on **alientek_rv1126b_defconfig**

```
make alientek_rv1126b_defconfig ARCH=arm64
```

Open menuconfig and modify

```
make menuconfig
```

Finally, save the modifications made in menuconfig to configs/alientek_rv1126b_defconfig.

```
scripts/kconfig/conf --savedefconfig=configs/alientek_rv1126b_defconfig Kconfig
```

You can also use the following command to save the modifications made in menuconfig.

```
make savedefconfig
```

```
mv defconfig configs/alientek_rv1126b_defconfig
```

3.3 Kernel Development

For users, kernel development often involves modifying the device tree and defconfig. Here we explain which device tree and defconfig users need to modify. Note that content related to driver development can be found in the driver development documentation.

3.3.1 How to Modify the Defconfig

According to the configuration file selected in Section 2.3 and the detailed explanation of the board-level configuration file in Section 3.1.2, the kernel defconfig file is SDK/kernel/arch/arm64/configs/alientek_rv1126b_defconfig.

The modification method is very simple. You can directly use ./build.sh to open menuconfig, and the modifications will be automatically saved to SDK/kernel/arch/arm64/configs/alientek_rv1126b_defconfig after completion. Except for U-Boot, which does not use ./build.sh to configure defconfig, all other components can use build.sh to save configurations. It is strongly recommended to use build.sh to modify defconfig to maintain consistency.

```
./build.sh kernel-config
```

```
./build.sh kconfig # Or this one.
```

3.3.2 Modifying the DTS

According to the configuration file selected in Section 2.3 and the detailed explanation of the board-level configuration file in Section 3.1.2, users must modify the DTS corresponding to the selected board-level configuration file. Different board-level configuration files may use different DTS files. For example, the device tree used by the ATK-DLRV1126B development board is as follows.

```
ls arch/arm64/boot/dts/rockchip/rv1126b-alientek*.dts
```

```
alientek@alientek:~/atk_dlr1126b_linux6.1_sdk_release_v1.0.0_20251202/kernel$ ls arch/arm64/boot/dts/rockchip/rv1126b-alientek*.dts
arch/arm64/boot/dts/rockchip/rv1126b-alientek.dtsi
arch/arm64/boot/dts/rockchip/rv1126b-alientek-mipi1080x1920-amp-mcu.dts
arch/arm64/boot/dts/rockchip/rv1126b-alientek-mipi1080x1920.dts
arch/arm64/boot/dts/rockchip/rv1126b-alientek-mipi720x1280-amp-mcu.dts
arch/arm64/boot/dts/rockchip/rv1126b-alientek-mipi720x1280.dts
arch/arm64/boot/dts/rockchip/rv1126b-alientek-mipi800x1280-amp-mcu.dts
arch/arm64/boot/dts/rockchip/rv1126b-alientek-mipi800x1280.dts
arch/arm64/boot/dts/rockchip/rv1126b-alientek-nodisplay.dts
```

For example, if the user selects 02_atk_dlv1126b_mipi720x1280_defconfig, as clearly stated in Section 3.1.2, the DTS used is rv1126b-alientek-mipi720x1280.dts. Users can add, delete, modify, or **rename** based on this DTS.

3.4 Buildroot Development

For Buildroot development, we usually configure the corresponding defconfig to customize the system. For detailed usage of Buildroot, please refer to the documents in the cloud disk:

10. User Manual/02 Auxiliary Documents/【ALIENTEK】Buildroot User Manual (Chinese translated by ALIENTEK) and 【ALIENTEK】Buildroot Quick Start Guide.

3.4.1 How to Modify the Defconfig

According to the configuration file selected in Section 2.3 and the detailed explanation of the board-level configuration file in Section 3.1.2, we know that the defconfig file for Buildroot is SDK/buildroot/configs/alientek_rv1126b_defconfig.

The modification method is very simple. You can directly use ./build.sh to open menuconfig, and the modifications will be automatically saved to SDK/buildroot/configs/alientek_rv1126b_defconfig after completion. It is strongly recommended to use build.sh to modify the defconfig to maintain consistency.

```
./build.sh buildroot-config  
./build.sh bconfig # Or this.
```

3.4.2 How to Add Custom Firmware

For users, the ALIENTEK SDK is already fully functional. If you only want to add your own firmware to the system, here are two methods:

Method 1:

For storing user data or applications, they are generally placed in the /userdata partition of the system. You can pre-place your firmware in the userdata image so that the user data/apps are included when flashing.

The path is:

SDK/device/rockchip/common/extra-parts/userdata/normal

You can delete all contents under normal, then place your apps into this folder. Run build.sh again, and it will generate SDK/rockdev/userdata.img.

Method 2:

Add firmware to the rootfs partition.

Place your custom firmware in:

SDK/buildroot/board/alientek/atk-dlv1126b/fs-overlay

This fs-overlay directory acts as the root directory. For example, the default ALIENTEK UI is located at fs-overlay/opt/ui.

Note: ALIENTEK UI is placed under /opt/ui for convenience. It is recommended that you do NOT place your files under /opt, because /opt belongs to the ROOTFS partition, while /userdata is a separate partition. Even if the userdata partition becomes full, it will NOT affect system booting!

Each time Buildroot is compiled, files here are copied into the filesystem. Do NOT delete existing files here, otherwise the system may lose functions.

Run build.sh again to generate SDK/rockdev/rootfs.img. Finally, flash update.img.

3.5 How to Flash the Firmware

Firmware refers to the complete system image, which is the packaged update.img generated from U-Boot, kernel, rootfs, and other partition image files.

Please refer to the Firmware Flashing Reference Manual in the cloud disk under:

10. User Manual / 02 Auxiliary Documents.

Flash the update.img file to the storage device.